Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)
Prof. Preet Kanwal (Associate Professor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

Activation Functions

Activation functions

- An activation function is a **non-linear transformation** that we apply on our input before propagating it to the next layer of neurons.
- Put simply, they decide whether a neuron should be activated or not.

$$Y = \text{Activation}(\Sigma(weight * input) + bias)$$

- Imagine a neural network without the activation functions. In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases.
- Although linear transformations make the neural network simpler, this network would be less powerful and will not be able to learn the complex patterns from data.
- A neural network without an activation function is essentially just a linear regression model.
- Thus we use a non linear transformation to the inputs of the neuron and this non-linearity in the network is introduced by an activation function.

Activation functions

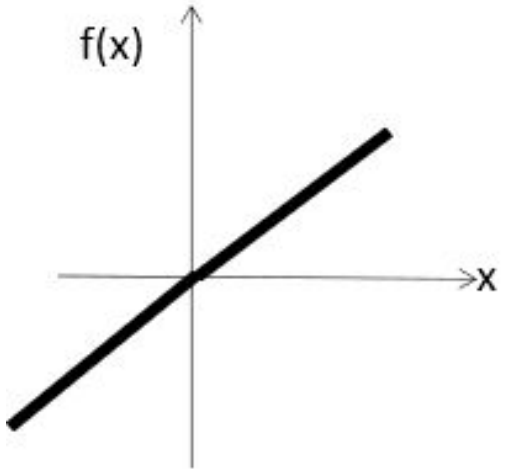
There are a number of options available:

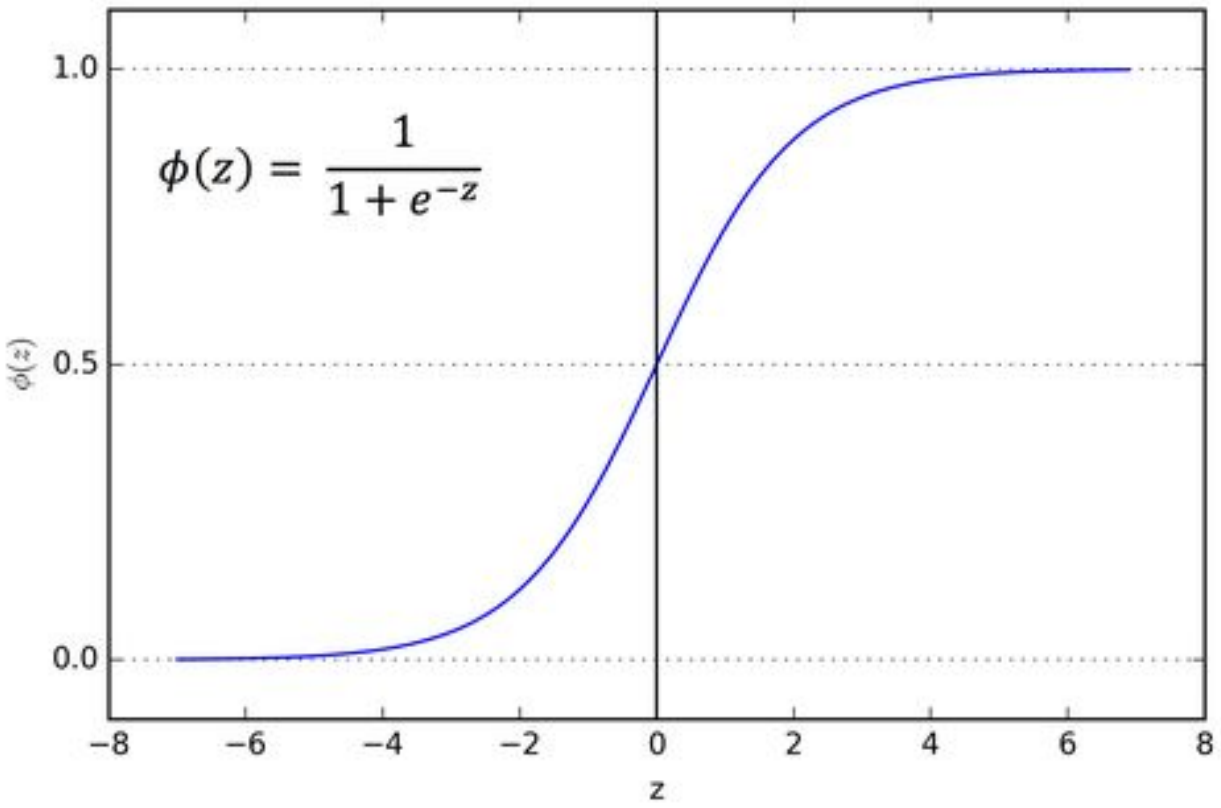
- Sigmoid
- tanh
- ReLU
- Softmax

Sigmoid

- **Equation :** $$f(x) = \frac{1}{1 + e^{-x}}$$

- **Range :** [0, 1]

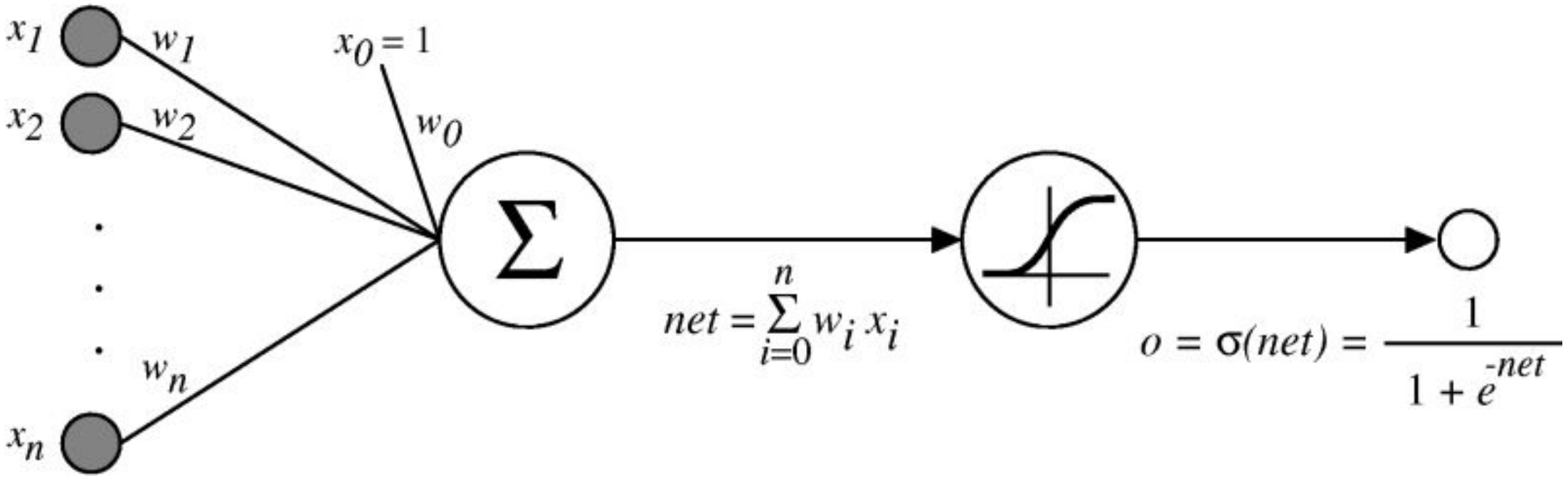




Example of an activation function -- Sigmoid

- While there are many activation functions, **sigmoid** is commonly used when # of hidden layers = 1.

$$f(x) = \frac{1}{1 + e^{-x}}$$



Sigmoid

- The function is **differentiable**.
- One of the most commonly used activation functions.

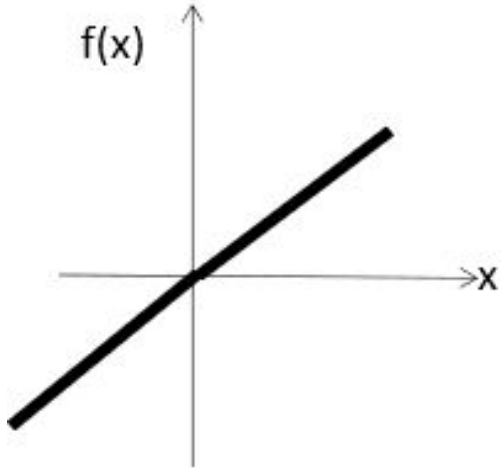
We know the range of sigmoid function is [0, 1]. What else do you know of that lies between 0 and 1?

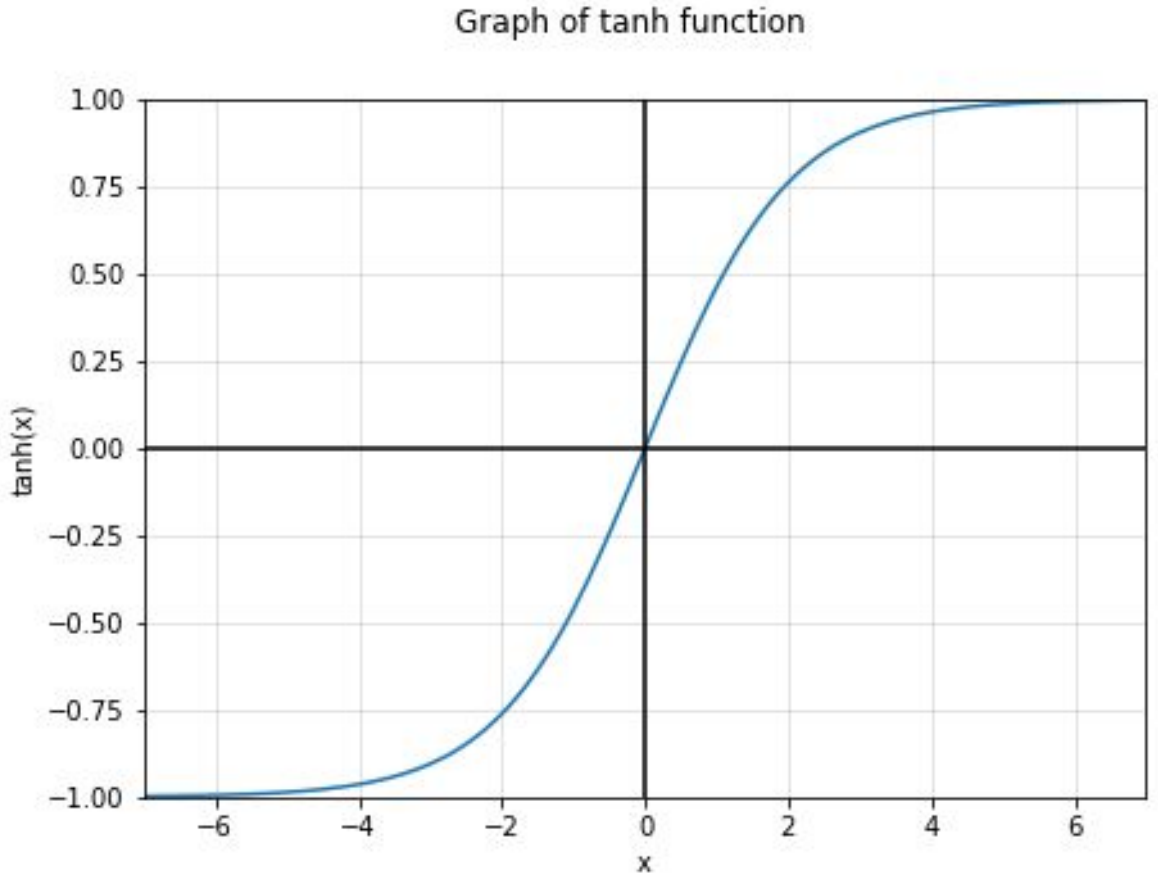
Probability.
Thus, sigmoid is used for models where we have to predict the probability as an output as we know the probability of anything exists only in [0, 1].

tanh (hyperbolic tangent)

- **Equation :** $f(x) = \dfrac{1 - e^{-2x}}{1 + e^{-2x}}$

- **Range :** [-1, 1]





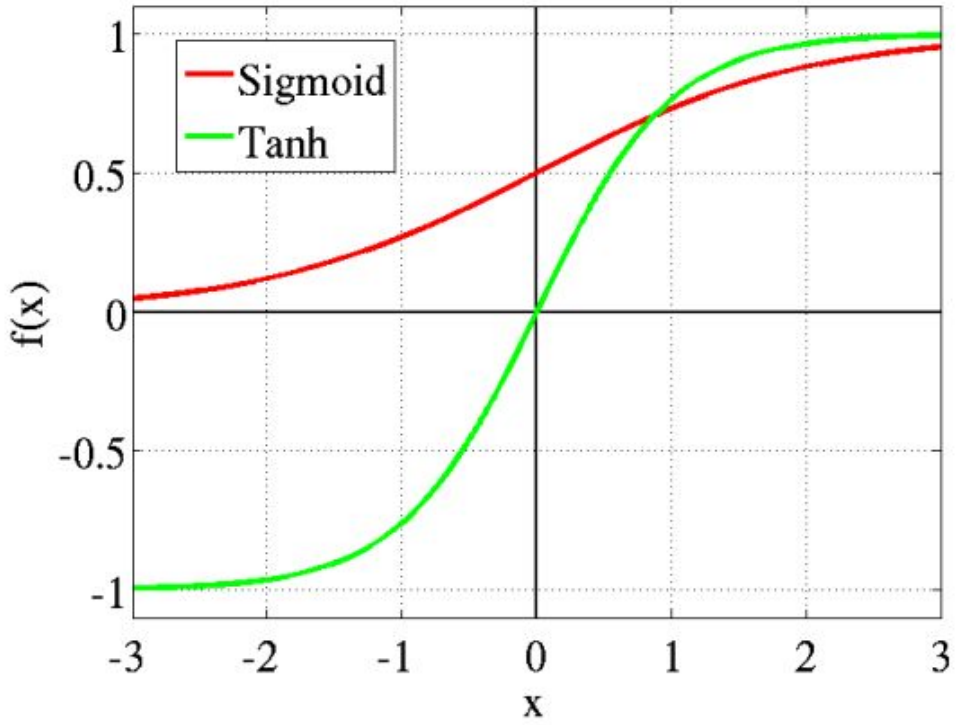
tanh

- The function is **differentiable**.
- The tanh function is mainly used for classification between two classes.

Comparing tanh with sigmoid

- tanh is also like sigmoid but better.

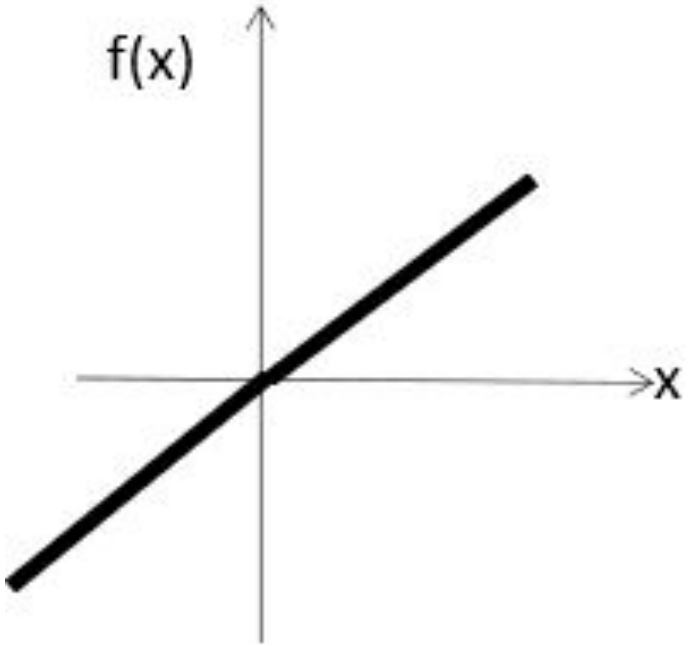
- The advantage being that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph.

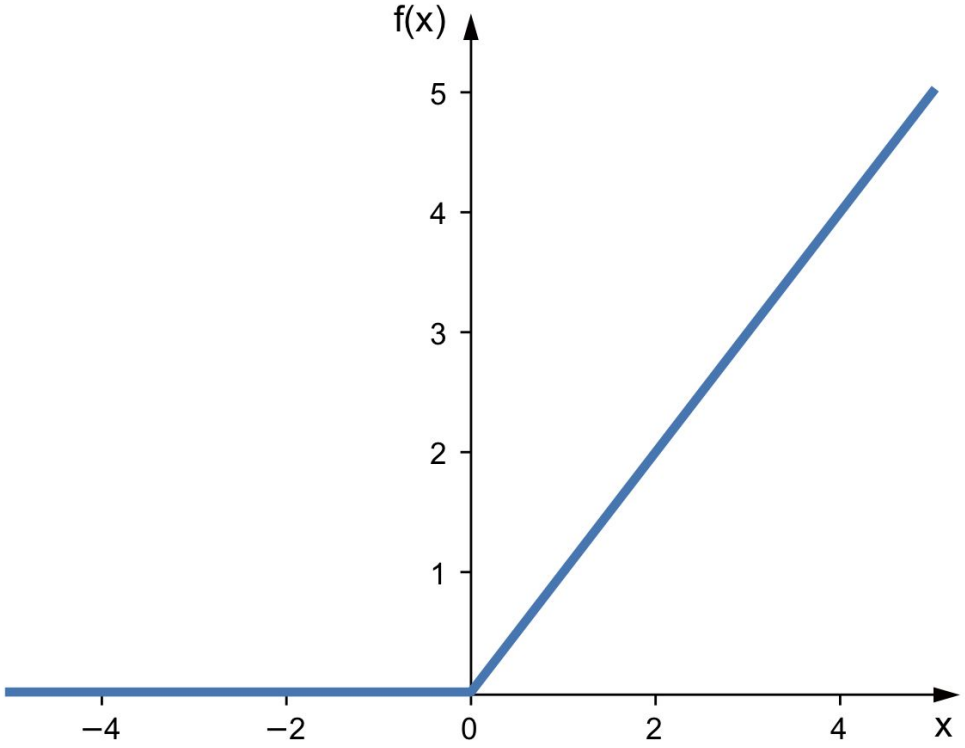


ReLU (Rectified Linear Unit)

- **Equation :**
  f(x) = x when **(x > 0)**
  = 0 when **(x <= 0)**

- **Range :** [0, infinity)





ReLU

- Very commonly used activation function in deep neural networks.

What is dying ReLU?

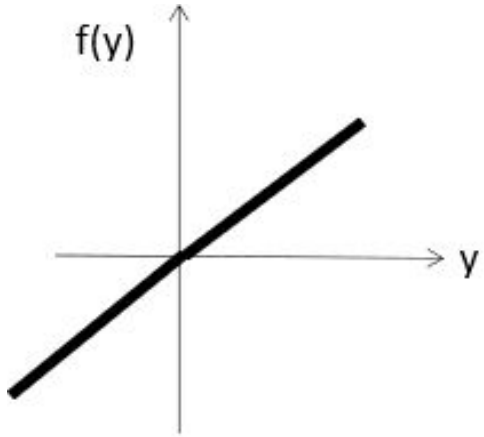
One issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.

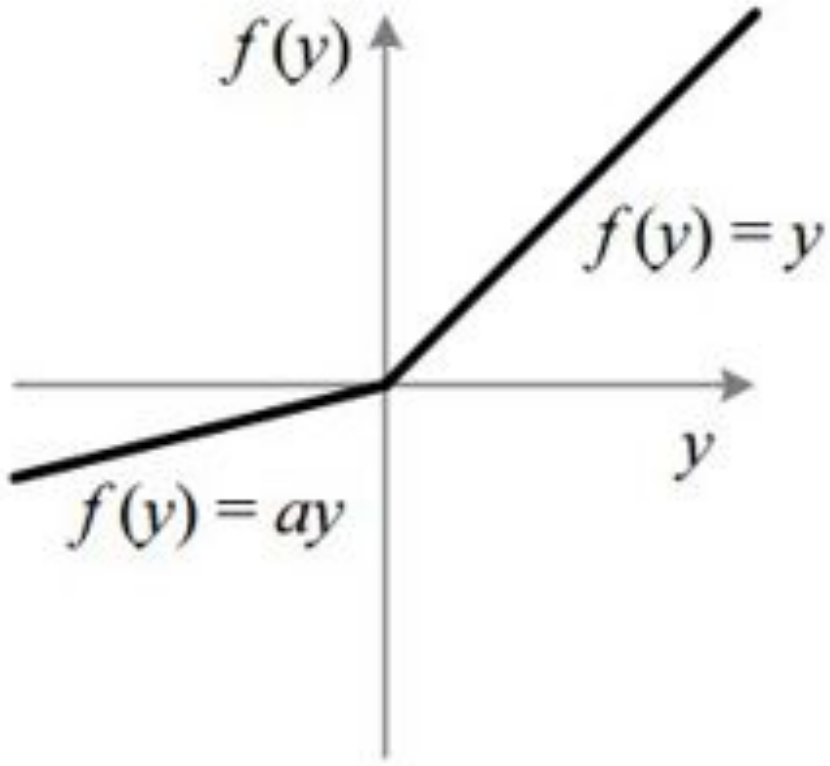
That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

How to deal with dying ReLU? Leaky ReLU

- **Equation :**
  f(y) = y when **(y > 0)**
  = ay when **(y <= 0)**

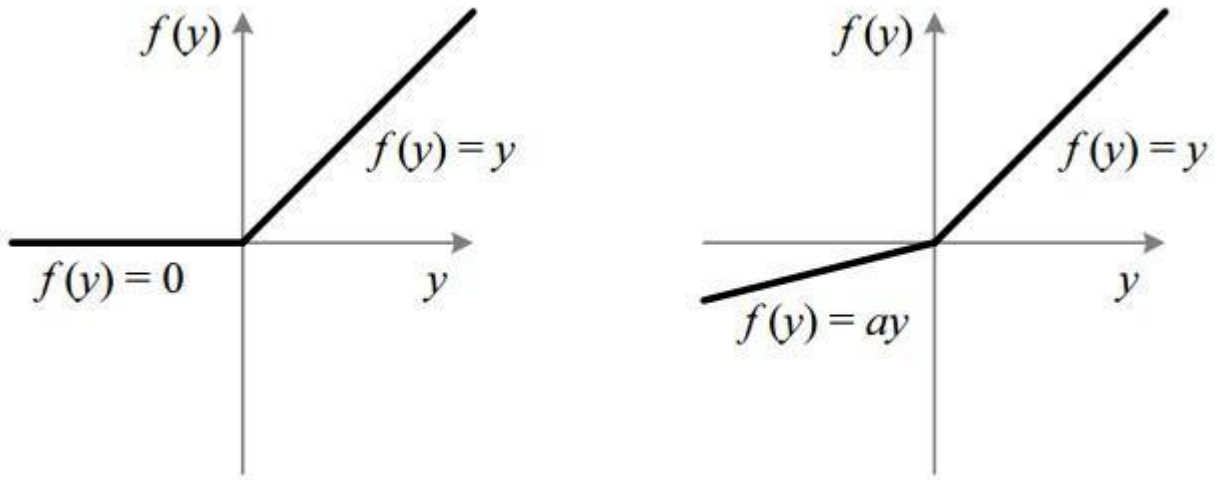
- **Range :** [-infinity, infinity)





"Leaky" ReLU

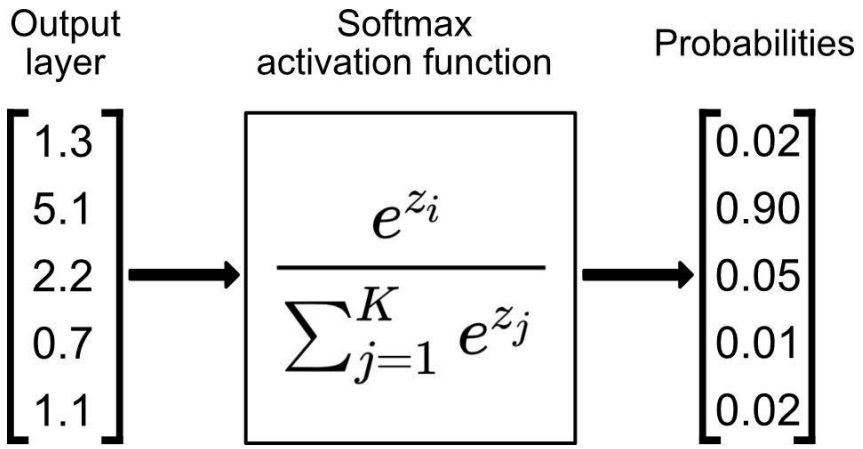
- The leak helps to increase the range of the ReLU function. Usually, the value of **a = 0.01**.
- Instead of setting some default value of a, we can set them as a parameter of a neural network and the network can be trained to learn the optimal value for it!



ReLU vs Leaky ReLU

Softmax

- It is a generalization of the sigmoid function i.e. when the number of classes is two, it becomes the same as the sigmoid function.
- **Used in the final layer** of an ANN while performing multi-class classification tasks.
- The Softmax activation function calculates the relative probabilities of each class for being the output. Thus, the sum of the softmax values will always be equal to 1.



z represents the values from the neurons of the output layer.

Which activation functions to use?

- It would be incredibly difficult to recommend an activation function that works for all use cases. Basically, there is no concept of “one size fits all.”

- Some points to be weary of:
  1. We are no strangers to the fact that calculus slows things down. Hence, we must consider how difficult it is to compute the derivative, if at all it is differentiable.
  2. How quickly a network with the chosen activation function converges.
  3. How smooth it is.
  4. Whether it satisfies the conditions of the Universal Approximation Theorem.

- It does not make sense to consider so many points before making a choice, hence we follow certain general guidelines.

Guidelines

- Sigmoid works better in case of binary classification problems. Continues to be popular with 1 hidden layer network.
- ReLU is widely used in deep neural networks as it yields better results.
- If we encounter a case of dead neurons in our network, leaky ReLU is the best choice.
- ReLU should only be used in the hidden layers. It simply sets everything that's negative to zero.

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Practice Questions

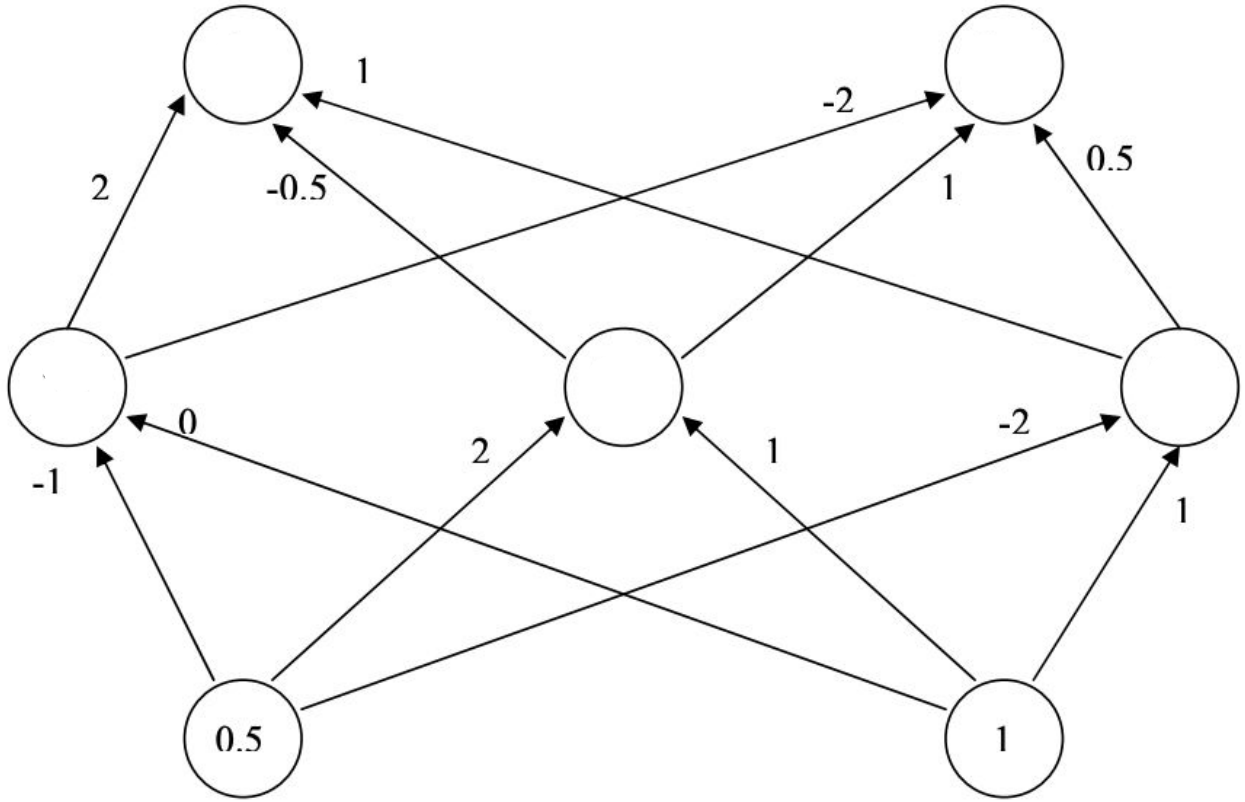
Practice Questions

Question on Linear Neurons:

The following is a network of linear neurons, that is, neurons whose output is identical to their net input, oi = neti. The numbers in the circles indicate the output of a neuron, and the numbers at connections indicate the value of the corresponding weight.

a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.

b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.

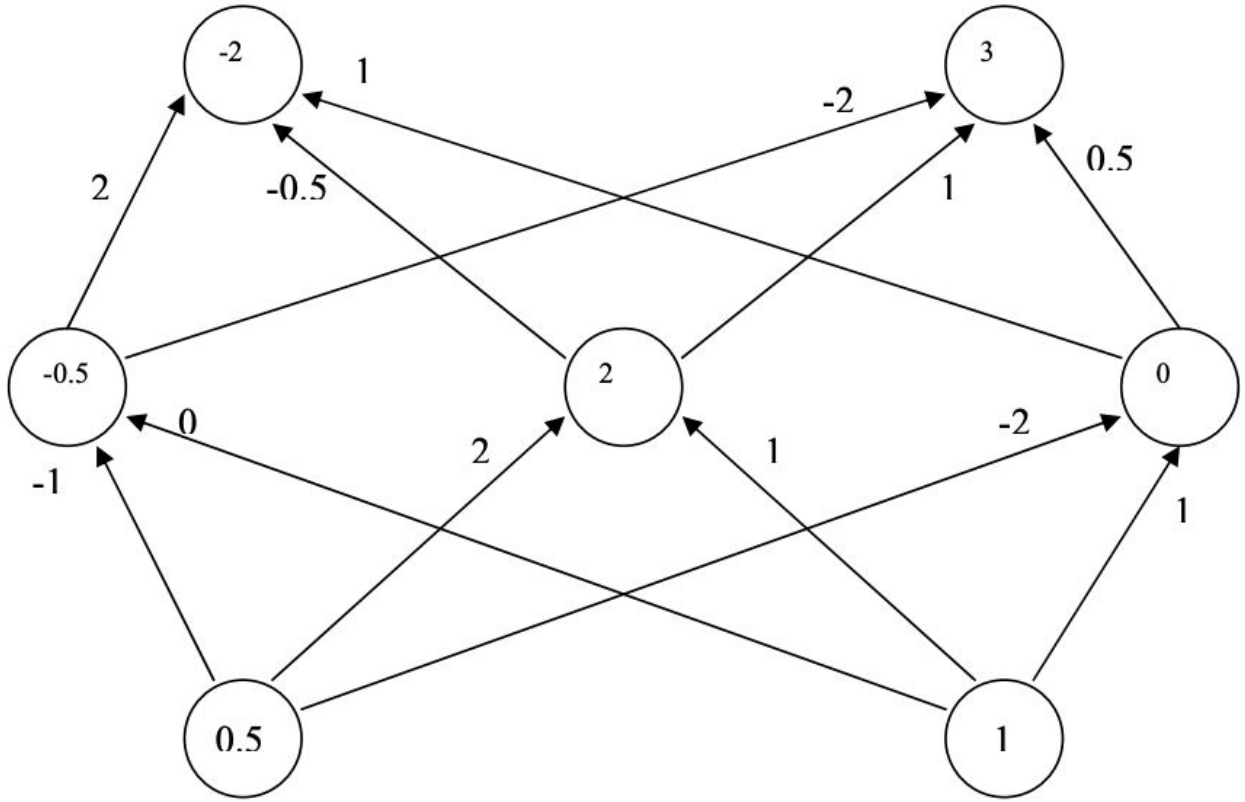


Practice Questions

a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.
Result: (-2, 3).

b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.

It's (-4, 6). We do not have to do all the computations, because every neuron computes a linear function on its inputs, which means that the entire network computes a linear function. For such a function, if we double the input, we simply double the output as well.



Practice Questions

A perceptron with a unipolar step function has two inputs with weights $w_1 = 0.5$ and $w_2 = -0.2$, and a threshold $\theta = 0.3$ (θ can therefore be considered as a weight for an extra input which is always set to -1). For a given training example $\mathbf{x} = [0, 1]^T$, the desired output is 0 (zero). Does the perceptron give the correct answer (that is, is the actual output the same as the desired output)?

A. Yes.

B. No.

Unipolar or Binary step function:- This function can be defined as,

$$f(x) = \begin{cases} 1, & if \quad x \geq \theta \\ 0, & if \quad x < \theta \end{cases}$$

where θ represents the threshold value. This function is most widely used in single layer nets to convert the net input to an output that is binary (1 or 0).

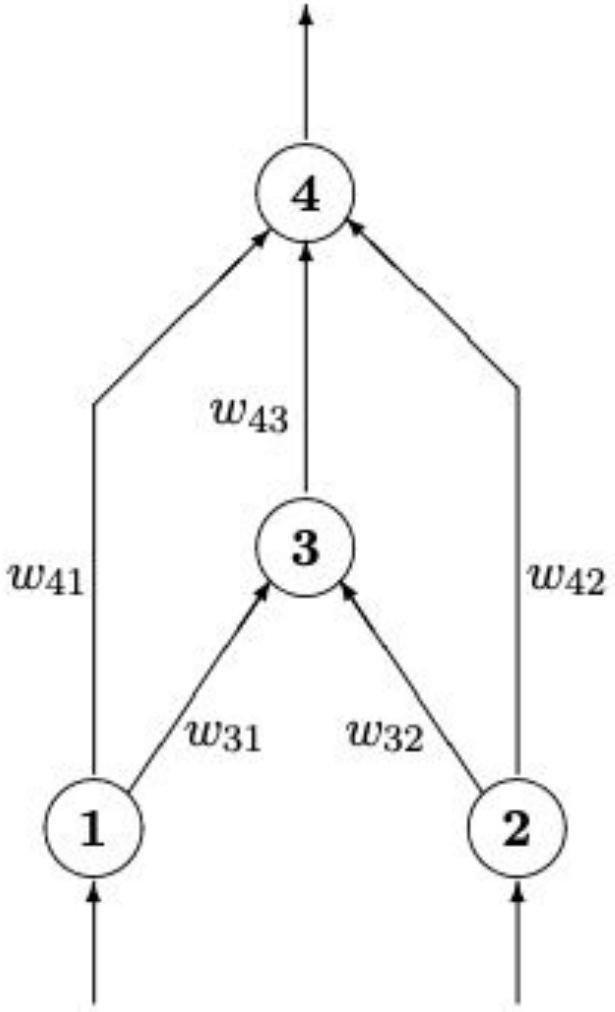
Practice Questions

The following network is a multi-layer perceptron, where all of the units have binary inputs (0 or 1) and binary outputs (0 or 1).

The weights for this network are w31 = 1, w32 = 1, w41 = −1, w42 = −1 and w43 = 3.

The threshold of the hidden unit (3) is 1.5 and the threshold of the output unit (4) is −0.5.

The threshold of both input units (1 and 2) is 0.5, so the output of these units is exactly the same as the input. Which of the following Boolean functions can be computed by this network?
AND, OR, XOR or none?



References

Sigmoid Neuron, Gradient Descent : http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Backpropagation : https://www.youtube.com/watch?v=XN5IRqtFhOY

Numerical : https://www.youtube.com/watch?v=0e0z28wAWfg

Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu